

Abstract—The Adobe Creative Jam for North America focuses on project creation utilizing Adobe’s Premiere Rush. The final product for submission in the contest is a short film utilizing Rush, to be submitted and judged by Adobe. The target of this proposal is to collect successful film projects, compare these projects, and discover patterns that help propel projects to success, then using these patterns determine a strategy and structured plan for the project’s submission. Here is the repository link and information: https://github.com/aceydufresne/cs4412-Plotting_Plots-Success, and <https://www.kaggle.com/datasets/thedevastator/rotten-tomatoes-top-movies-ratings-and-technical>.

Specifically searching for what successful films contain, that less successful films do not, or at least contain less of, essentially the disjunction of the set of success and the set of non-successful. The final output, or result for this pattern searching should be the attributes and aspects of cinema that affect the outcome the most significantly. In this way, we should be able to create a rough ‘script’/outline for the contest, or a very abstracted prediction of what the most potentially successful project we could build.

I. M2 SUMMARIZATION:

In the initial implementation, I covered the embedded vectors, specifically finding the Cosine Similarity between cinematic narratives, with limited success. We found that within specific genres, critic reviews tended to be fairly similar, as shown in the plot below: However, this did not translate into plot embeddings,

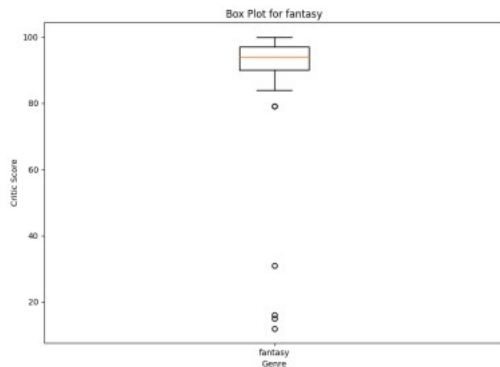


Figure 5. Updated Fantasy Genre.

Figure 1. Similarity Within Genres.

or the similarity between the story beats. We found

the embedded vectors, by creating a window and sliding it across the plot descriptions. The issue we

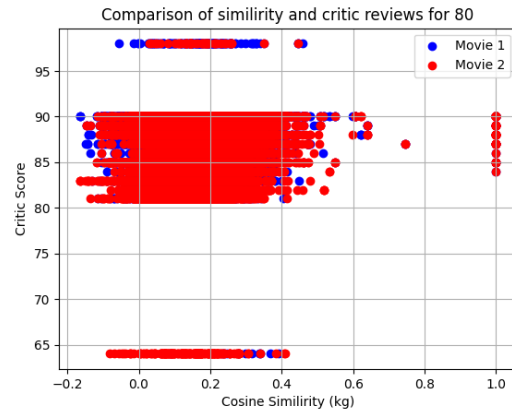


Figure 2. Similarity Of Vectors.

ran into, was that the vector embeddings were too dense, which we attempted to deal with using various dimensional reduction techniques, which did not see much success. Next attempting to cluster the vectors, using K-means, the issue here, was that the clusters were too grouped together to gather accurate results: Refinement: After the M2 assignment, I have been

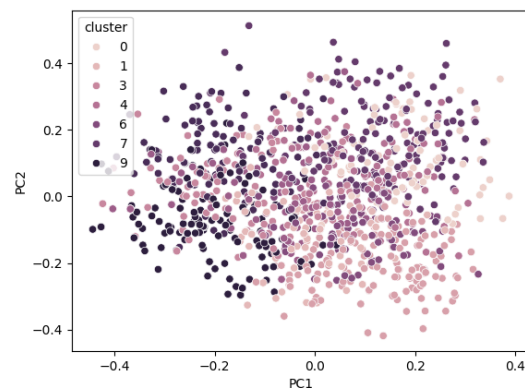


Figure 3. K-Means Clustering.

experimenting with better methods of embedding the vectors. I worked with my Natural Language Processing professor, and worked out a strategy. The method I worked out was to simplify the plots, using generic string pattern concepts. So instead of embedding the plots directly, we find the generic meaning of the

narratives. For example, character does something, or something happens to character. Then using a dataset of common names or similes for character, we can embed characters as either, main protagonists, side-characters, or antagonists, in the hope to reduce the complexity of the narratives. Similarly, we can use a neural network to understand the meaning of what is happening to our characters, then simplify that down, that way with simpler vectors, the cosine scoring will be much more effective, as well as more time consuming.

II. M3: IMPLEMENTATION

As you will notice and encounter in this section, the assignment is much less dense in methodology and verbose in wording than the M2 assignment. This is because every step of this portion of the implementation was image processing, which took considerably longer to accomplish than the previous section. Creating the dataset alone, took over ten hours, for each attempt. The first attempt, all the images in the dataset were corrupted, which means over fifteen hours were wasted. The first step was building the dataset, a recap from the M1 assignment:

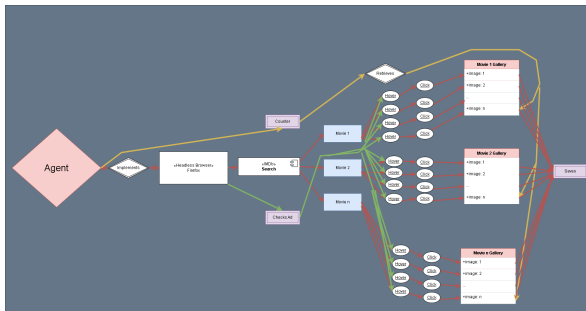


Figure 4. K-Means Clustering.

For M2, I implemented a datascraper that collects the gallery for each film in the IMDb website, for each corresponding film in the initial Kaggle dataset, then applied some pre-processing, resizing and reshaping the images, to increase efficiency. The patterns we are looking for in this visual portion of the dataset, are several film theory/color theory rules and principles: *Contrast/Color Theory *60-30-10 Rule *Rule of Thirds *Framing

A. 60-30-10

The '60-30-10' Rule is a design rule, used in graphic design, interior decoration, and most importantly for us: cinematic settings. In this rule, sixty percent of a room should be a dominant color, the secondary color should make up thirty percent, and ten percent should be made up of an accent color. Below is a notable example, from 'LaLa Land,' which is included in our dataset, As can be seen the primary



Figure 5. K-Means Clustering.

color, is the dominant dark purple, with a blue-ish gray acting as the floor, and the accent is the yellow in the dress. This is a great example to tackle how our program processes an image. The first thing we need to do, is to resize the image, to a fifty by fifty image, we can downscale by quite a lot, because in this step, we don't care about the details, and if a color disappears from an image after down-scaling, then that color would not have been included in the rule to begin with. After down-sizing, we use k-means to cluster pixels to similar colors. As we can see in the example image, although the sixty percent, dark purple, is the pre-dominant color, or the background, the entire background is not the same hue. So what we need to do, is group similar colors together, then using a threshold value, we want to merge those colors together, that are similar.

After which, we have a new 'image,' with blotches of dominant colors. What we do now is iterate through the image, and check what color a pixel should be grouped with, in this way we can find the size, or more importantly the weight of a color. After which, we can divide the cluster size by twenty-five-thousand, or fifty times fifty, because every image is now at a standard size. We can then take the ratios of each of the majority colors, just like the PCA in the last assignment, there will be some dimensions that can't be accounted for, but in those cases, we already know, they won't fall under the 60-30-10 rule.

We iterate through each films independent image dataset, keeping track of how many scenes, or images from each film contain a 60-30-10 composition, then we can normalize it compared to how many images are in that dataset, to get a final score for each film. Which we can plot against the critic scores of our initial dataset: Before we start analyzing the scatter plot, we should take into account, we are already aware this graph will not tell us much about the patterns going, for example, we are comparing well composited scenes against critic scores, but only plotting one type of visually pleasing scene. Even still, the results are a bit disheartening, as there seems

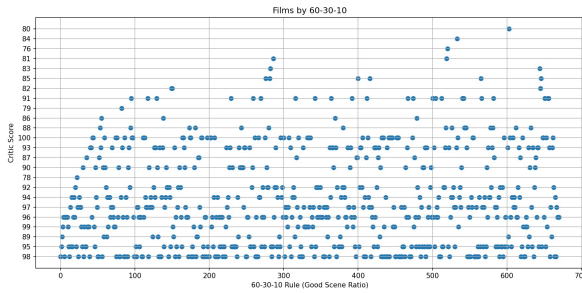


Figure 6. K-Means Clustering.

to be absolutely no correlation.

B. Contrast:

For the next visual component/attribute we want to examine, or process, is the contrast, or general color theory. Again the steps are fairly similar to before, resize the images, use k-means to cluster pixels together. Then find the range, max color value minus minimum color value. Again the results are disappointing:

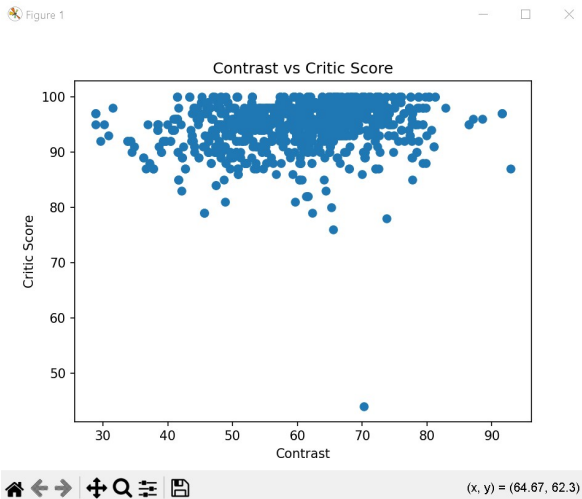


Figure 7. K-Means Clustering.

At first glance, it seems we have a strong grouping, but upon closer inspection we see no real pattern between contrast/brightness. All these graphs will be detailed further at the synopsis for the assignment. But for now, we can say on a surface level, there is no pattern between the three attributes.

C. Rule of Thirds:

The rule of thirds is a fairly abstract concept, where when a grid is placed over a frame, objects are either framed by the grid, or ontop of the grid. Which can better be understood with an example: The subject of the image is between the lines, this is fairly difficult to implement, the way I am attempting this is by first using a sobel edge detector, then comparing those

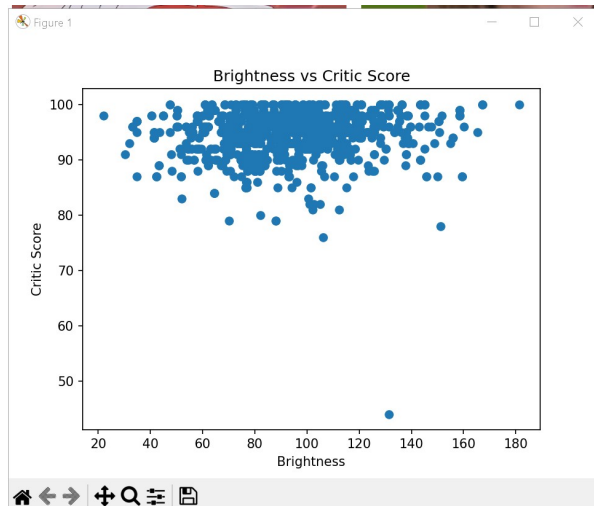


Figure 8. K-Means Clustering.

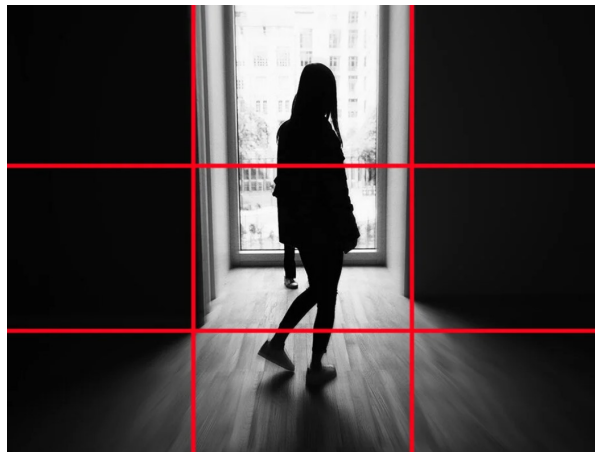


Figure 9. Rule of Thirds.

edges to a grid, and assigning a score. Which we can then use a threshold to either pass an image or not. The second component, is Google's YOLO, as well as a image classifier. Specifically we want to look for faces, and compare the positioning within the grid.

III. SYNOPSIS:

The implementation has been slow for this portion, because every step takes an absorbent amount of time, as is the nature of image processing. While in this implementation I was able to produce various graphs for the attributes, I believe the actual patterns will be discovered when applying the Apriori Rule to all the components, and we can obtain clearer plots. Right now the plan is to use a tree structure, such that an image will be graded on each rule, in sequence, which should reduce the noise in the scatter plots. For example, if a film has low contrast, high rule of thirds, and high 60-30-10, we can start to speed how we examine these patterns.